

# Enunciado de la práctica (Nivel 5)

**Objetivo:** Aprender a generar y usar certificados digitales en Linux con OpenSSL.

**Escenario:**

El servidor genera su propio certificado autofirmado.

El cliente recibe ese certificado y lo usa para verificar la identidad del servidor.

## Pasos de la práctica

### EN EL SERVIDOR

Generar la clave privada RSA (2048 bits):

```
openssl genpkey -algorithm RSA -out server.key -pkeyopt rsa_keygen_bits:2048
```

Crear un certificado autofirmado (válido 365 días):

```
openssl req -new -x509 -key server.key -out server.crt -days 365
```

Te pedirá datos como nombre de dominio (ej: servidor.local). **No necesitas registrar un dominio real.**

Puedes añadirlo en /etc/hosts para que el cliente resuelva ese nombre a la IP de tu servidor:

```
sudo nano /etc/hosts
```

Y añadir:

```
192.168.1.50    servidor.local
```

Envía el certificado. Comando

```
scp server.crt cliente@IP:/ruta/destino
```

o mediante USB, email, etc.

Qué es /etc/hosts

Es un archivo del sistema operativo que asocia nombres de host (como servidor.local) a direcciones IP (como 192.168.1.50).

Antes de que un sistema consulte un servidor DNS, revisa este archivo para ver si hay alguna entrada que coincida.

```
192.168.1.50    servidor.local
```

**192.168.1.50** → IP del servidor en tu red local.

**servidor.local** → nombre que quieres usar para conectarte al servidor.

Esto significa que cada vez que tu computadora intente conectarse a servidor.local, automáticamente usará la IP 192.168.1.50.

No necesita consultar un servidor DNS externo.

Ahora el servidor tiene:

server.key → clave privada.

server.crt → certificado (contiene la clave pública + identidad + firma).

### EN EL CLIENTE

**Recibir el certificado del servidor** (server.crt) mediante scp, ssh o incluso USB.

**Ver el contenido del certificado:**

```
openssl x509 -in server.crt -text -noout
```

Aquí se ve: clave pública, validez, nombre del servidor.

YA TENEMOS:

- Generación de clave privada y certificado autofirmado (server.key + server.crt).
- Envío del certificado al cliente.
- Visualización del contenido del certificado en el cliente (openssl x509 -text).

A REALIZAR: Comunicación cifrada entre cliente y servidor

Importante Leer:

En esta práctica trabajaremos con cifrado híbrido combinando **AES (simétrico)** y **RSA (asimétrico)** para proteger la comunicación entre un cliente y un servidor. Aunque en ejercicios iniciales es común generar la clave AES en **formato hexadecimal**, `openssl rand -hex 32 > clave_aes.txt`, más fácil de visualizar y manipular como texto en laboratorio, en esta práctica avanzamos a la versión **binaria** para generar la clave AES, `openssl rand -out clave_aes.bin 32`

Esto permite usar la clave directamente para cifrar los mensajes sin conversiones adicionales, que para nosotros son transparentes ya que lo realiza openssl,

La clave simétrica al cifrarla con la **clave pública del servidor** extraída de su certificado, se garantiza que solo el servidor pueda descifrarla.

Todos los mensajes posteriores se transmiten de forma segura usando AES, reflejando cómo se maneja la criptografía en escenarios reales, donde la protección del intercambio de claves es fundamental.

En cliente:

1.- Obtener la clave pública del servidor, cuando tienes el certificado del servidor (server.crt) en el cliente, **ya tienes la clave pública del servidor**, porque está incluida dentro del certificado.

Para **extraerla** en Linux con OpenSSL:

```
openssl x509 -in server.crt -pubkey -noout > server_pub.pem
```

- in server.crt → indica el certificado del que quieres extraer la clave.
- pubkey → extrae **solo la clave pública**.
- noout → no muestra otra información por pantalla.
- > server\_pub.pem → guarda la clave pública en un archivo para usarla después.

Después de esto, server\_pub.pem contiene la clave pública que puedes usar para **cifrar mensajes o verificar firmas**.

2.- El cliente genera una clave simétrica (AES) para cifrar los mensajes y la envía cifrada con la clave pública del servidor. Así, **el servidor es el único que puede descifrarla con su clave privada**.

**Comandos en Linux** usando OpenSSL:

# Generar clave AES de 256 bits

```
openssl rand -out clave_aes.bin 32
```

clave\_aes.bin → contiene la clave secreta que se usará para cifrar mensajes posteriores con AES.

3.- Cifrar la clave AES con la clave pública del servidor

```
openssl rsautl -encrypt -pubin -inkey server_pub.pem -in clave_aes.bin -out clave_aes.enc
```

- pubin -inkey server\_pub.pem → indica que estamos usando **clave pública** del servidor.
- in clave\_aes.bin → la clave AES que queremos enviar.
- out clave\_aes.enc → archivo que se enviará al servidor.

Solo el **servidor**, con su clave privada (server.key), podrá descifrar esta clave AES

4.- Enviar la clave AES cifrada al servidor

Por ejemplo, con scp:

```
scp clave_aes.enc usuario@IP_SERVIDOR:/home/usuario/
```

En el servidor:

En el servidor: descifrar la clave AES

```
openssl rsautl -decrypt -inkey server.key -in clave_aes.enc -out clave_aes.bin
```

- inkey server.key → la **clave privada** del servidor.
- in clave\_aes.enc → archivo recibido del cliente.
- out clave\_aes.bin → ahora el servidor tiene la misma clave AES que el cliente.

## 5.-Usar la clave AES para cifrar mensajes

Cliente cifra mensaje:

```
openssl enc -aes-256-cbc -salt -in mensaje.txt -out mensaje.enc -pass  
file:./clave_aes.bin
```

Servidor descifra mensaje:

```
openssl enc -d -aes-256-cbc -in mensaje.enc -out mensaje.txt -pass file:./cla-  
ve_aes.bin
```

Ahora **todos los mensajes posteriores** se cifran con AES (simétrico), usando la misma clave compartida de forma segura gracias a RSA.

## 6.- envío de mensajes.

### 6.1.- Enviar usando SCP (sencillo para prácticas locales)

```
scp mensaje.enc usuario@IP_SERVIDOR:/home/usuario/
```

usuario → nombre del usuario en la máquina servidor.

IP\_SERVIDOR → IP del servidor (ej: 192.168.1.50).

/home/usuario/ → ruta donde quieres dejar el mensaje.

Luego, en el servidor:

```
openssl enc -d -aes-256-cbc -in mensaje.enc -out mensaje.txt -pass file:./cla-  
ve_aes.bin
```

Esto descifra el mensaje usando la clave AES que ya compartieron.

## **Documentación de la Práctica: Intercambio de Información Certificado**

### **Enunciado de la práctica. Breve**

#### **Entorno de trabajo (Linux)**

Sistema operativo usado (ej: Debian 12, Ubuntu 22.04, etc.).

Configuración de red (adaptador puente, IPs asignadas).

Usuarios creados (ej: cliente y servidor).

#### **4. Procedimiento paso a paso**

Separado por roles: Servidor - Cliente

#### **5. Comandos utilizados**

Listar todos los comandos ejecutados (openssl, scp, etc.).

Explicar brevemente qué hace cada uno.

#### **6. Evidencias (capturas de pantalla)**

Añadir capturas de pantalla a tu documentación

#### **7. Conclusiones y dificultades (cómo se resolvieron)**

#### **8. Anexo (opcional)**

Posibles mejoras:

Automatización con script Bash.